StreetCare AI — logical architecture

What the system does, independent of technology: who uses it, the controls every request passes, the use cases, the rules and AI agents behind them, and the information they keep

Logical view — technology is shown in the technical and deployment architectures

1 Actors

Residents
outside the council
report street defects through Website · StreetCare app · Contact centre → the open queue

Street Care Coordinator
C · triage
scans the queue, studies reports, opens an AI-prepared review audits reviews and AI calls

Highways Inspector
I · 2 per district
decides reviews for own district; never one they opened (four-eyes) also opens reviews and audits

Customer Services Adviser
A · residents' line
answers residents: report status and documented rules only; never a likelihood or crew hours

uses pages

2 Presentation

Sign in — *everyone* — C I A
Report queue — *rank · filter by type* — C I
Report detail — *screening · evidence · estimates* — C I
Decision queue — *reviews awaiting my district* — I
Audit trail — *reviews · AI calls · chat* — C I
Street Care chat — *ask the documents / records* — C I A

every request

3 Access & control

`401` **Identity** — signed in, session live
`403` **Permission** — role allows this operation
`422` **Validation** — request well formed
`404` **Existence** — report / review is known
`403` **District** — inspector appointed there
`409` **Case rules** — one review per revision, one decision, decider ≠ opener, decision open for the assessment

allowed only

4 Use cases

Sign in / out — start or end a session pages + permissions follow from the role — *who* C I A
Browse the queue — 40 open reports, ranked by rules; filter by defect type; open one report — *who* C I
Study a report — standards screening, street works, evidence, two estimates — no AI — *who* C I
Ask Street Care — classify the question; answer from documents or records, by role — *who* C I A
Audit — reviews, decisions, AI calls with tokens and latency, chat turns — *who* C I
Open a review — 5 AI analyses on the facts → recommendation → pause for a decision — *who* C I
Decide a review — checks, then resume: documents for the decision → recorded — *who* I

applies rules

5 Rules & AI agents

Business rules — deterministic, explainable, testable

Report rules — references, units, spellings, latest revision
Standards screening — each requirement: Met / Not met / Not documented
Street-works check — live and conflicting permits, coordination triggers
Evidence analysis — sentence types; where the documents disagree
Outcome estimates — repeat likelihood + band, crew hours, overrun
Knowledge retrieval — standard sections and passages, for citation
Decision rules — decisions open, district, four-eyes, once only
Disclosure rules — what each role may hear; street figures ≥ 5 reports

facts

AI agents — interpret the facts, compute nothing

one task each · fixed labelled answer fields · every call logged

1 **Planner** → 2 **Requirements analyst** → 3 **Evidence analyst** → 4 **Risk analyst** → 5 **Review writer**

Human decision gate — the review waits, durably, for an inspector
6 **Order writer** — *ORDER · INSPECT*
7 **Resident-update writer** — *ORDER only*
Street Care assistant — *chat · 1 call*

reads · writes

6 Information

Open reports — reports, attached documents (the live queue)
Streets & works — districts, streets, street-works permits
Policy knowledge — 16 standards, guides and policies, in sections
History & models — past reports and repairs; 3 trained models
Review case files — review, recommendation, decision, documents
Paused reviews — the state each review waits in at the gate
Audit log — every AI call; every chat question and answer
Staff & sessions — 16 accounts, roles, districts, sessions

prompts

Language-model service — external
external · one call per agent task
writes prose from given facts; never decides, never computes

Decision outcomes

what each inspector decision produces

ORDER REPAIR
works order → the council's repair contractor + resident update
2 AI documents

INSPECT ON SITE
inspection request → the inspector who will visit the defect
1 AI document

REFER FOR COORDINATION
coordination brief → the network coordination team assembled by rules
0 AI documents

resident update: plain words, no staff name

The main flow — one report, from queue to resident

1 **C** opens a review → 2 **rules** compute the facts → 3 **agents 1–5** analyse, recommend → 4 **gate** review waits → 5 **I** decides (checked) → 6 **agents 6–7** draft documents → 7 **record** outcome + audit → 8 **A** tells the resident

Actor · Presentation · Rule-based · AI-assisted · Human decision / case record · Prediction & history · Information · C Coordinator · I Inspector · A Adviser

StreetCare AI — data architecture: entity-relationship model

Left: the business entities in the delivered files and reference data · Right: the physical tables the service writes, with every column and type — crow's-foot notation

Built from data/*.csv · config/config.py · persistence/models.py · checkpoint.db schema

A Source & reference model *conceptual — CSV files, config and PDFs*

District *config*

PK	district_name	4 districts
	streets	12 each
	inspectors	2 appointed

Street *48 streets*

PK	street_name	Station Road
FK	district	→ District
	street_category	Main route …

StreetWorksPermit *register*

PK	permit_reference	SWP0071263
FK	street_name	→ Street
	works_location	
	works_promoter	
	works_description	
	permit_status	Live / Cancelled

R1 District 1 — N Street · R2 Street 1 — 0..N StreetWorksPermit · R3 Street 1 — 0..N DefectReport

DefectType *5 types*

PK	defect_type	Footway …
FK	standard_document	→ PDF
	requirement kinds	per standard

DefectReport *history + open queue*

PK	report_reference	SC0052158
PK	revision_number	latest kept
	report_channel · reporter_type	
FK	street_name	→ Street
	district · street_category	copied
	location_description	
	location_fixed · photograph_attached	Yes / No
FK	defect_type	→ DefectType
FK	linked_defect_type	0..1 second
	defect_depth_mm · defect_width_mm	mm
	water_depth_mm · water_reach_m	mm · m
	water_location · overhang_location	
	clear_height_m · on_highway_land	
	lighting_fault · column_number_given	
	column_lean_degrees · column_height_m	
	previous_reports_at_location	
	planned_crew_hours	h
	live_ / conflicting_street_works	history
	review_outcome · repeat_cause	history
	reported_again	target
	supporting_document_count	open only

RepairLog *1 per report*

PF	report_reference	→ report
	repair_method	
	traffic_management	
	crew_size	
	planned_crew_hours	h
	completion_code	
	actual_crew_hours	target
	reported_again	

R4 DefectType 1 — 0..N DefectReport · R5 DefectType 1 — 1..N Requirement · R6 DefectType 1 — 1 PolicyDocument · R7 DefectReport 1 — 0..1 RepairLog · R8 DefectReport 1 — 0..N ReportDocument

Requirement *10 kinds*

PK	requirement_kind	depth …
FK	defect_type	→ DefectType
	requirement_name	
	requirement_sentence	quoted
	rule figures	exact decimals

ReportDocument *attached*

PK	document_id	DOC-841228
FK	report_reference	→ report
	document_type	3 kinds
	document_title	
	document_text	

PolicyDocument *16 PDFs*

PK	document_name	*.pdf
	kind	standard · guide …
	numbered sections	1 … n

same document types

StatementSentence *corpus*

PK	sentence_id	
	report_reference	own refs
	document_type	
	sentence_number	
	sentence_text	
	sentence_length_characters	
	sentence_type	target · 9

R9 PolicyDocument 1 — 1..N KnowledgePassage

KnowledgePassage *205 in ChromaDB*

PK	id	
FK	document_name	→ PolicyDocument
	section_number	
	passage_number	order
	carried_characters	overlap
	document (text)	≤ 700 chars
	embedding	384-d MiniLM

Relationships

R1	**District 1 — N Street**	12 streets per district (INSPECTION_DISTRICT_STREETS)
R2	**Street 1 — 0..N StreetWorksPermit**	street_name; Live permits count, Cancelled do not
R3	**Street 1 — 0..N DefectReport**	street_name; the street fixes the deciding district
R4	**DefectType 1 — 0..N DefectReport**	defect_type, and again (dashed) via linked_defect_type
R5	**DefectType 1 — 1..N Requirement**	parsed from that type's standard PDF at run time
R6	**DefectType 1 — 1 PolicyDocument**	its Defect-Standard-<type>.pdf
R7	**DefectReport 1 — 0..1 RepairLog**	report_reference; the log holds actual crew hours
R8	**DefectReport 1 — 0..N ReportDocument**	report_reference; 60 documents for 40 open reports
R9	**PolicyDocument 1 — 1..N KnowledgePassage**	document_name + section_number for citations

Checked on the data

- report_reference is unique in every clean file once the latest revision is kept (highest revision 3)
- 99.3 % of repair-log references also appear in report history — R7 is optional, not mandatory
- statement sentences are a separate labelled corpus: 6.1 per reference on average (max 15), none in history
- live queue: 40 open reports, 60 documents, 28 permits (23 live) on 48 streets in 4 districts

Model inputs: DefectReport + counts screened under each Requirement + trigger flags; targets reported_again, actual_crew_hours, sentence_type.

B Operational model *physical — database/streetcare.db (SQLAlchemy) and database/checkpoint.db (LangGraph)*

user_accounts *7 cols · 16 seeded*

PK	username	VARCHAR
	display_name	VARCHAR
	role	VARCHAR
	district	VARCHAR NULL
	password_hash	VARCHAR
	password_salt	VARCHAR
UQ	session_token	VARCHAR NULL

opens

agent_runs *7 cols*

PK	run_id	VARCHAR
UQ	thread_id	VARCHAR
FK	coordinator_id	VARCHAR
	report_reference	VARCHAR
	coordinator_request	TEXT
	status	VARCHAR
	llm_call_count	INTEGER

logs 5–7 calls

llm_calls *8 cols*

PK	call_id	INTEGER
FK	run_id	VARCHAR
	agent_name	VARCHAR
	model_name	VARCHAR
	prompt_tokens	INTEGER
	response_tokens	INTEGER
	total_tokens	INTEGER
	latency_milliseconds	INTEGER

asks

chat_turns *12 cols*

PK	turn_id	INTEGER
FK	username	VARCHAR
	user_role	VARCHAR
	question_text	TEXT
	question_type	VARCHAR
	report_reference	VARCHAR NULL
	street_name	VARCHAR NULL
	defect_type	VARCHAR NULL
	answer_text	TEXT
	source_documents	TEXT
	closest_distance	FLOAT
	was_refused	BOOLEAN

decides (decided_by)

report_reviews *37 cols*

PK	review_id	VARCHAR
UQ	run_id	VARCHAR
	report_reference	VARCHAR
	revision_number	INTEGER
	defect_type	VARCHAR
	linked_defect_type	VARCHAR
	street_name	VARCHAR
	district	VARCHAR NULL
	coordinator_request	TEXT
	requirements	JSON
	requirements_met_count	INTEGER
	requirements_not_met_count	INTEGER
	requirements_not_documented_count	INT
	live_permits	JSON
	conflicting_permits	JSON
	coordination_triggers	JSON
	assessment	VARCHAR
	decisions_open	JSON
	evidence	JSON
	disagreements	JSON
	repeat_likelihood	FLOAT
	likelihood_band	VARCHAR
	expected_crew_hours	FLOAT
	planned_crew_hours	FLOAT NULL
	overrun_expected	BOOLEAN NULL
	crew_hours_text	TEXT
	review_plan_text	TEXT
	requirements_analysis_text	TEXT
	evidence_review_text	TEXT
	prediction_analysis_text	TEXT
	recommendation_text	TEXT
	recommended_decision	VARCHAR
	status	VARCHAR
	order_document_text	TEXT NULL
	document_details	JSON NULL
	resident_update_text	TEXT NULL
	coordination_brief	JSON NULL

thread_id: one paused / finished run

review_decisions *8 cols*

PK	decision_id	INTEGER
UQ	review_id	VARCHAR
FK	run_id	VARCHAR
	decision	VARCHAR
FK	decided_by	VARCHAR
	decided_by_role	VARCHAR
	decided_by_district	VARCHAR
	decision_reason	TEXT

database/checkpoint.db

checkpoints *7 cols*

PK	thread_id	TEXT
PK	checkpoint_ns	TEXT
PK	checkpoint_id	TEXT
	parent_checkpoint_id	TEXT
	type	TEXT
	checkpoint	BLOB
	metadata	BLOB

writes *9 cols*

PK	thread_id	TEXT
PK	checkpoint_ns	TEXT
PK	checkpoint_id	TEXT
PK	task_id	TEXT
PK	idx	INTEGER
	task_path	TEXT
	channel	TEXT
	type	TEXT
	value	BLOB

Status columns — allowed transitions

agent_runs.status

RUNNING → AWAITING_DECISION → COMPLETE

FAILED (502)

report_reviews.status

AWAITING_DECISION → REPAIR_ORDERED | INSPECTION_REQUESTED

REFERRED_FOR_COORDINATION

one decision only

Relationships and bridges to model A

- **user_accounts 1 — 0..N agent_runs** — coordinator_id = username (the coordinator who opened it)
- **agent_runs 1 — 1 report_reviews** — run_id, unique in both; joined inner in fetch_review_records
- **report_reviews 1 — 0..1 review_decisions** — review_id, unique; joined outer, so undecided reviews show
- **agent_runs 1 — 5..7 llm_calls** — run_id; 5 before the gate, then 7 ORDER · 6 INSPECT · 5 REFER
- **user_accounts 1 — 0..N review_decisions** — decided_by = username; never the opener (four-eyes)
- **user_accounts 1 — 0..N chat_turns** — username; every reply stored, refusals too
- **agent_runs 1 — 1 checkpoint thread** — thread_id = checkpoints.thread_id = writes.thread_id
- **report_reviews → DefectReport (A)** — (report_reference, revision_number): one review per revision
- **review_decisions → District (A)** — decided_by_district must hold the report's street
- **chat_turns → DefectReport · Street · DefectType · PolicyDocument (A)** — report_reference, street_name, defect_type; source_documents names the PDFs

No FOREIGN KEY constraints are declared: persistence/repository.py, the only module that queries, joins on these keys. Grey dots link to model A.

exactly one · zero or one · one or many · zero or many · PK primary key · PF primary + foreign key · UQ unique · FK logical reference · related, not a key

access · review & decision · LLM-written / logged · reference / run · labelled training data

StreetCare AI — data architecture: data flow and lineage

Where every dataset comes from, how it is cleaned, what it becomes and which keys join it — row counts, scores and sizes are read from the project's own files

Built from config/config.py · data_pipeline/ · modeling/ · knowledge/ · persistence/models.py

1 Sources · data/

Historical extracts — training & held-out evaluation

five CSVs as delivered: mixed units, three reference notations, stray spellings, duplicates

file	rows	cols	target	used for
report_history_train.csv	32,738	31	reported_again	repeat likelihood
report_history_eval.csv	8,202	31	reported_again	held out
repair_logs_train.csv	26,095	27	actual_crew_hours	expected crew hours
repair_logs_eval.csv	6,476	27	actual_crew_hours	held out
statement_sentences.csv	36,661	8	sentence_type	sentence type (9 classes)

Live inputs — data/input/

the open queue the service works on

file	rows × cols	after cleaning
open_defect_reports.csv	45 × 27	40 open
report_documents.csv	62 × 6	60 docs
street_works_register.csv	29 × 6	23 live

40 reports = latest revision of each reference
28 permits, 23 live · documents keyed by document_id
no outcome columns: nothing to leak

Policy documents — data/docs/

16 PDFs, numbered sections

- 5 Defect-Standard-*.pdf
- 8 Guide-*.pdf (photo, measure, …)
- 1 Highways-Policy.pdf
- 1 Report-Rules.pdf
- 1 Past-Report-Decisions-Casebook.pdf

casebook is indexed but never quoted in chat

2 Preparation · offline

run_data_pipeline.py · data_pipeline/

1. load every column as text · NULL_TOKENS → missing
2. drop exact duplicates and rows missing the target
3. SC-52158 · sc52158 · SC0052158 → one reference
4. fold category spellings to one canonical value
5. cm · inch · ft · min → mm · m · h (Decimal, half-up)
6. below minimum / above maximum → missing
7. keep the latest revision of each report
8. text: markup, entities, whitespace, CAPS → plain

rows received → kept

history train	32,738 → 30,569 (–2,169)
history eval	8,202 → 7,648 (–554)
logs train	26,095 → 25,537 (–558)
logs eval	6,476 → 6,337 (–139)
sentences	36,661 → 35,717 (–944)

dark = kept · light = removed

Live cleaning — at service start

tools/report_data_tools.py

the same cleaners as training:
`report_cleaning · document_cleaning`
`register_cleaning`

then, per open report, in plain code:
screening under the parsed standard
street-works count (live / conflicting)
sentence evidence + disagreements
repeat likelihood + expected crew hours

run_knowledge_indexing.py

knowledge/

`pdf_extraction`
text of each of the 16 PDFs
`chunk_splitting`
numbered sections → passages of
700 chars, 120 carried over;
a requirement is never split
`vector_index`
embed, persist; read-back check
of every section

3 Derived assets

processed/

5 clean CSVs · same columns

report_history_*_clean
repair_logs_*_clean
statement_sentences_clean

`output/`
`data_quality_report.json`
gaps, units as written, non-
canonical values per column

run_model_training.py · feature_builder.py · modeling/

fit on train only · scored on the held-out eval file · saved only if all three pass

Repeat likelihood
LogisticRegression
reported_again · 27 features
30,569 train / 7,648 eval
ROC AUC 0.835
minimum 0.80 (red) · scale 0.5–1
Brier 0.117

Expected crew hours
RandomForest · 300 trees
actual_crew_hours · 20 features
25,537 train / 6,337 eval
R² 0.902
minimum 0.86 (red) · scale 0.5–1
MAE 0.42 h

Sentence type
TF-IDF + LogisticRegression
sentence_type · 9 classes
28,573 train / 7,144 eval
macro-F1 0.978
minimum 0.95 (red) · scale 0.5–1
accuracy 0.977

report_queue

in memory
40 rows, built
once at import

ranked by code,
never by a model

+ details per
report

database/chroma_store/

ChromaDB · chroma.sqlite3
collection street_care_documents
205 passages · all-MiniLM-L6-v2
cosine distance
metadata: document_name,
section_number, passage_number,
carried_characters

facts + model figures copied into the review at the gate

4 Operational stores · online

database/streetcare.db

SQLite via SQLAlchemy · persistence/repository.py is the only module that queries it · keys are joined in code (no FOREIGN KEY constraints)

user_accounts — 7 cols · 16 seeded

PK	username	
	display_name · role	
	district	*inspectors only*
	password_hash	*PBKDF2-SHA256 ×200k*
	password_salt	*16 bytes*
UQ	session_token	*24 bytes · NULL = out*

1 : N username

chat_turns — 12 cols · every reply

PK	turn_id	
FK	username · user_role	→ username
	question_text · question_type	
	answer_text · source_documents	refusals too
	closest_distance · was_refused	

1 : N

agent_runs — 7 cols · one per review run

PK	run_id	*RUN-<12 hex>*
UQ	thread_id	*= checkpoint thread*
FK	coordinator_id	→ username
	report_reference	
	coordinator_request	
	status	*RUNNING → AWAITING → COMPLETE*
	llm_call_count	*5, then 7 / 6 / 5*

1 : 1 thread_id

checkpoint.db — LangGraph SqliteSaver

	checkpoints · writes	*2 tables*
FK	thread_id	→ agent_runs
	ReportReviewState	*paused at the gate*
	resume →	*order · update · record*

1 : 1

report_reviews — 37 cols · one per revision

PK	review_id	*REV-<12 hex>*
UQ	run_id	→ agent_runs
	report_reference · revision_number	
	defect_type · street_name …	*identity*
	requirements · met/not met/not doc	*screening*
	live / conflicting_permits	*register*
	coordination_triggers · assessment	
	evidence · disagreements	*JSON*
	repeat_likelihood · band	*model*
	expected / planned_crew_hours	*model*
	5 agent texts · recommended_decision	*LLM*
	status	*AWAITING → decided*
	order_document_text · details	*LLM 6*
	resident_update_text	*LLM 7*
	coordination_brief	*code (REFER)*

1 : 0..1

review_decisions — 8 cols · 0..1 per review

PK	decision_id	*autoincrement*
UQ	review_id	→ report_reviews
FK	run_id	→ agent_runs
	decision	*ORDER / INSPECT / REFER*
FK	decided_by · _role · _district	→ username
	decision_reason	*≥ 10 chars*

decided_by → user_accounts

1 : 5–7

llm_calls — 8 cols · 5–7 per run

PK	call_id	*autoincrement*
FK	run_id	→ agent_runs
	agent_name · model_name	
	prompt / response / total_tokens	
	latency_milliseconds	*accepted replies only*

Keys that join the data

`report_reference`
every CSV, documents, sentences,
runs, reviews, chat_turns · SC0052158

`revision_number`
latest kept; one review per
report revision

`street_name`
reports ↔ street_works_register ↔
district (INSPECTION_DISTRICT_STREETS)

`document_id`
report_documents ↔ evidence
findings ↔ disagreements

`run_id · thread_id`
agent_runs ↔ report_reviews ↔
llm_calls ↔ checkpoint.db

`review_id`
report_reviews ↔ review_decisions

`username`
user_accounts ↔ coordinator_id,
decided_by, chat_turns

Data rules enforced in code

Exact decimals
every unit conversion uses Decimal, ROUND_HALF_UP

No leakage
fit on train only; eval held out; train-vs-eval gap kept

Same cleaning
live open reports pass the training cleaners

Facts before prose
agents see tool-computed facts; they compute nothing

Need-to-know
advisers never see a likelihood or crew hours

Small groups
a street figure needs ≥ 5 reports behind it

Full audit
every LLM call and every chat turn is stored

Credentials
salted PBKDF2 hashes; tokens cleared on sign-out

One review
per report revision; a decision only once

Footprint on disk now

megabytes, from du on the Project folder

data/	17 MB
processed/	17 MB
output/models/	13 MB
chroma_store/	2.4 MB
checkpoint.db (+WAL)	0.74 MB
streetcare.db	0.13 MB

the live database stays small: text and JSON only

Read top to bottom: raw files are cleaned once offline into processed files, models and an index; the running service cleans the live queue the same way and writes every review, decision, LLM call and chat turn to SQLite.

PK primary key · UQ unique · FK logical reference joined in code. Counts: received → kept from output/data_quality_report.json; scores from output/model_metrics.json.

Raw / stored data · Deterministic code · Trained ML model · LLM-written data · Review & decision records · Access data · data flow · lineage into a record

StreetCare AI — technical architecture

The technology in each tier, the exact library versions in the project environment, the modules that use them, and how the tiers talk to each other

Versions from the project venv (myenv) · settings from config/config.py

1 Client tier

Web browser
HTML · JS · WebSocket

renders Streamlit pages;
no logic, no local storage

Streamlit — 1.67
run_dashboard.py · dashboard/ (6 page modules)

st.navigation + st.Page — only the pages in the role's list
st.session_state keeps the user and token on the server side
st.form · st.spinner · st.dataframe · st.badge · .streamlit/config.toml theme

dashboard/api_client.py — httpx 0.28
the single HTTP client

`call_service(method, path, query, payload) → JSON`
adds X-Session-Token · timeout 300 s · spinner for long calls
401 → clears the session, back to sign-in · ≥ 400 → st.error(detail)

REST · JSON · X-Session-Token · :8081

2 API tier

Uvicorn — 0.54
ASGI server

`0.0.0.0:8081`
`run_api_service.py`
checks 13 required files
before importing routes

FastAPI — 0.162
api/routes.py · service_application

public_router
`/auth/signin · /auth/signout`
guarded_router
`9 routes · Depends(authorize_request)`
report_locks: one threading.Lock per open report
handlers → 502: ValueError · genai APIError · httpx.HTTPError

Pydantic — 2.14
api/schemas.py

4 request models: SignIn, Review,
Decision, Chat · 8 response models
Literal[...] enums built from config lists
StringConstraints: strip, min / max length
→ 422 before any handler runs

access/ — stdlib
passwords · sessions · permissions

`hashlib.pbkdf2_hmac('sha256', 200 000)`
16-byte salt · hmac.compare_digest
`secrets.token_urlsafe(24)`
`PERMISSION_BY_ROUTE → 403`
`authorize_district → 403`

invoke / resume · *ask*

3 Orchestration & AI

LangGraph — 1.2
agentic/graph.py · nodes.py · state.py

`StateGraph(ReportReviewState)`
`TypedDict · llm_call_count: Annotated[int, operator.add]`
10 nodes · 7 add_edge chains · 2 add_conditional_edges
`route_report_decision → ORDER / INSPECT / REFER`
`interrupt(review_id) → invoke(Command(resume=decision))`
nodes return only the keys they write

SqliteSaver — ckpt-sqlite 3.1
langgraph-checkpoint-sqlite

`sqlite3.connect(checkpoint.db,`
`check_same_thread=False)`
a checkpoint after every node
resumable after a restart
or from another process

google-genai — 3.0
agentic/language_model.py · agents.py

`genai.Client · key from .env`
temperature 0.2 · max 8 192 tokens
automatic function calling off
`parse_labelled_response`
→ fixed labelled fields or error
each reply → llm_calls (tokens, ms)

chat/
Street Care assistant

`conversation_memory`
the asker's own last turns
`street_care_assistant`
classify · retrieve · answer
`record_answer`
report facts by role, no LLM

facts for each node

4 Domain, ML & knowledge

tools/ — plain Python
deterministic — never calls the LLM

`report_data_tools.py`
`screening_tools.py`
`evidence_tools.py`
`prediction_tools.py`
`retrieval_tools.py`
`decision_tools.py`
`review_document_tools.py`

scikit-learn — 1.9
modeling/ · data_pipeline/feature_builder.py

`Pipeline(ColumnTransformer(`
`numbers: SimpleImputer(median) → StandardScaler`
`categories: SimpleImputer(most_frequent) → OneHotEncoder))`
LogisticRegression (max_iter 3000) → repeat likelihood
RandomForestRegressor (300 trees, leaf 10, features 0.7) → crew hours
TfidfVectorizer → LogisticRegression → 9 sentence types
joblib 1.6 (compress 3) · pandas 3.1 · numpy 2.5 · seed 1313

ChromaDB + sentence-transformers — 1.5 · 6.2
knowledge/

pypdf 7.0 → page text → regex section headings
passages of 700 chars with 120 carried; a requirement is never split
PersistentClient · collection street_care_documents · cosine
all-MiniLM-L6-v2 embeddings on CPU (torch 2.15) · 384 dimensions
exact section read-back for citations; casebook excluded from chat
`standard_reading`
requirement figures parsed from each standard as Decimal

5 Persistence

SQLAlchemy — 2.1
persistence/ — the only module that queries

declarative_base · 6 ORM models (models.py)
create_engine(sqlite:///database/streetcare.db, check_same_thread=False)
repository.py: 13 functions · select + join / outerjoin

streetcare.db
SQLite · 6 tables
users · runs · reviews
decisions · calls · chat

checkpoint.db
SQLite · WAL
checkpoints · writes
paused review state

chroma.sqlite3
ChromaDB
205 passages + vectors
16 PDFs

*.pkl · *.json
output/models/
3 fitted pipelines
2 feature schemas

processed/*.csv
pandas
5 clean extracts
+ quality report

6 Build, test & run

Python — 3.14 · 3.11
local venv · Docker image

decimal (ROUND_HALF_UP)
threading · secrets · hmac
python-dotenv 1.2

pytest — 9.2
tests.py

50 tests: data, ML, RAG, agents,
API and dashboard
stand-in model replaces Gemini

Entry points
run from Project/

`run_data_pipeline · run_model_training · run_knowledge_indexing`
`run_database_setup · run_api_service · run_dashboard`
each checks the files it needs first

Docker
python:3.11-slim

one image, two processes
EXPOSE 8080
entrypoint script

GitHub Actions
.github/workflows/build.yml

SonarCloud quality scan
platform trigger on push

Interfaces & protocols

Browser ↔ Streamlit
HTTP + WebSocket · :8080

Streamlit → FastAPI
REST · JSON · :8081

header X-Session-Token

FastAPI → LangGraph
in-process invoke / resume

LangGraph → Gemini
HTTPS · google-genai SDK

chat → Gemini
HTTPS · 1 call per question

tools → ChromaDB
in-process · SQLite file

tools → models
joblib.load at import

all → database
SQLAlchemy ORM sessions

graph → checkpoints
shared sqlite3 connection

first embedding
Hugging Face Hub download

Cross-cutting concerns

Configuration
config/config.py: one source
for every value; .env for the key

Security
PBKDF2 hashes · random tokens
route → permission → district

Errors
HTTPException 401/403/404/409/422
one handler → 502, run FAILED

Concurrency
per-report lock; threads share
SQLite with check_same_thread off

Audit
llm_calls: tokens + latency
chat_turns: every reply

Determinism
temperature 0.2 · labelled fields
seed 1313 · Decimal half-up

Cost bound
≤ 7 LLM calls per review
≤ 1 per chat question

Client · Service code · Security · LLM / orchestration · Machine learning · Storage · Tooling · call / data flow · LLM call

StreetCare AI — deployment architecture

One Docker image, one container, two processes: Streamlit is the only published port; the REST API, its stores and the models live inside the container

Built from Project/Dockerfile · docker-entrypoint.sh · .github/workflows/build.yml · config/config.py

1 Access & network

Staff roles
three roles, a browser and a sign-in

- **Street Care Coordinator** — queue · opens reviews
- **Highways Inspector (2 / district)** — decides reviews
- **Customer Services Adviser** — status & chat only

uses

Web browser
any modern browser

`http://<host>:8080`

Streamlit pages over a websocket
token kept server-side in
`st.session_state`

:8080

Host or deployment platform
Docker engine · the platform may set PORT

```
docker build -t streetcare-ai .
docker run --rm -p 8080:8080 \
    --env-file .env -v streetcare-db:/app/database \
    streetcare-ai
```

README command + the two flags it needs

-p 8080:8080 → container :8080 (the only published port)

Google Gemini API
egress

external LLM · HTTPS via google-genai

model = GEMINI_MODEL from the env file
review start: 5 calls (one per agent)
decision: +2 ORDER · +1 INSPECT · +0 REFER
chat: at most 1 call per question

HTTPS · GEMINI_API_KEY

Hugging Face Hub
egress

embedding model download

`all-MiniLM-L6-v2`
fetched on the first embedding call
(knowledge search) unless already
in the container's model cache

first embedding call

2 Runtime — one container

Docker container `streetcare-ai` — `python:3.11-slim · WORKDIR /app · EXPOSE 8080 · ENTRYPOINT ./docker-entrypoint.sh`

Process 1 · Streamlit dashboard
`run_dashboard.py · 0.0.0.0:${PORT:-8080} · PID 1`

- pings the API's /docs first; if it does not answer shows 'The service is not answering…'
- sign-in page, then only the role's pages:
  Report queue · Report detail · Decision queue
  Audit trail · Street Care chat
- `dashboard/api_client.py` — the only HTTP client
- `.streamlit/config.toml`: theme, minimal toolbar, usage statistics off · holds no business rule

exec

docker-entrypoint.sh
sh · set -eu — the container's start command

```
1  python run_api_service.py &
     api_pid=$!   (background child)
2  exec streamlit run run_dashboard.py \
     --server.port=${PORT:-8080}
   trap INT TERM EXIT → kill $api_pid
```

starts

httpx → localhost:8081

X-Session-Token
timeout 300 s

Process 2 · FastAPI REST service
`run_api_service.py → uvicorn 0.0.0.0:8081 · not published`

Start-up gate
13 required files must exist:
data, processed CSVs, 3 models,
2 schemas, index, streetcare.db

Warm-up at import
build_workflow(checkpoint.db)
clean + screen 40 open reports once
one lock per report (in memory)

Guard · api/ + access/
session token → role → district
401 · 403 · 404 · 409 · 422 · 502

11 endpoints + OpenAPI
/auth · /queue · /reviews · /chat · /audit
docs at :8081/docs (internal)

LangGraph review workflow
10 nodes · 7 agents · interrupt() gate
SqliteSaver → checkpoint.db
survives a container restart*

Tools + 3 scikit-learn models
joblib .pkl loaded once, in memory
screening · register · evidence
likelihood · crew hours · sentences

Knowledge search
ChromaDB PersistentClient
all-MiniLM-L6-v2 on CPU
standards parsed at run time

Language-model gateway
agentic/language_model.py
temperature 0.2 · ≤ 8 192 tokens out
every call logged to llm_calls

R/W

Writable state `/app/database`
mount a volume — --rm deletes it otherwise

`streetcare.db`
SQLite · 6 tables:
users, runs, reviews,
decisions, llm_calls, chat

`checkpoint.db`
LangGraph SqliteSaver
paused review runs
WAL mode

`chroma_store/`
chroma.sqlite3 · 205 passages from 16 PDFs

read

Read-only artefacts
baked in by COPY . . at build time

`data/`	8 CSVs + 16 PDFs · 17 MB
`processed/`	5 clean CSVs · 17 MB
`output/models/`	3 .pkl + 2 schemas · 13 MB
`config/config.py`	every setting, no secrets

Ports and processes

`:8080`	**Streamlit (PID 1)**	published with -p
`:8081`	**FastAPI / uvicorn**	inside the container
`:8081/docs`	**OpenAPI page**	inside the container
`local run`	**same two ports**	two terminals, myenv

Secrets and settings
`.env`
GEMINI_API_KEY · GEMINI_MODEL · FRONTEND_PORT
`.dockerignore`
drops .env (and venvs, caches) from the image —
pass it at run time with --env-file
`config/config.py` holds every other value: ports, paths, limits

Start-up order
1. entrypoint starts the API in the background
2. API checks the 13 required files (exit 1 if any is missing)
3. imports routes: workflow, models, index; screens 40 reports
4. uvicorn listens on :8081
5. Streamlit serves :8080 — first page pings /docs

image streetcare-ai → docker run

3 Build & delivery

Team member
5 branches, one per
teammate · git push
or pull request

push

.github/workflows/build.yml
on: push to any branch · pull_request

```
1  actions/checkout@v2  (fetch-depth: 0)
2  SonarCloud scan  → SonarQube server
3  Trigger API  (if: always())
     POST {repo_id, branch_name, org_name}
```

runs on ubuntu-latest · quality scan only:
it builds no image and runs no tests

SonarQube
sonar.server.examly.io

code-quality dashboard
per branch

Assessment platform
API_URL from repo secrets

told which repo and branch
was pushed

Offline build — run once from Project/
produces every artefact the image copies

```
1  python run_data_pipeline.py
     → processed/*.csv · data_quality_report.json
2  python run_model_training.py
     → output/models/ · model_metrics.json
3  python run_knowledge_indexing.py → chroma_store/
4  python run_database_setup.py  → streetcare.db
5  pytest tests.py               → 50 tests
```

files

docker build -t streetcare-ai .
image recipe — Project/Dockerfile

```
FROM python:3.11-slim
pip install -r requirements.txt
  + CPU-only torch wheel index
COPY . .
  code + data + processed + models + index
RUN chmod +x docker-entrypoint.sh
EXPOSE 8080 · ENTRYPOINT the script
```

Deployment notes

Single replica. SQLite files and the per-report locks live in one process, so run exactly one container; scale up, not out.

Only :8080 is public. The REST API and /docs answer only inside the container; the dashboard is the sole way in.

Embedding package. requirements.txt does not list sentence-transformers, which the index's embedding function needs.

Long requests. Starting a review waits for 5 Gemini calls; the dashboard allows 300 s per request.

Keep /app/database. With --rm and no volume, reviews, decisions, checkpoints, chat audit and sessions vanish on stop.

Process supervision. Streamlit is PID 1; if the background API exits, the page shows 'service not answering'.

Offline hosts. Pre-download all-MiniLM-L6-v2 into the image; otherwise the first search needs Hugging Face.

CI scope. build.yml scans code quality only; the 50 tests run locally with pytest tests.py.

** a paused review resumes after a restart only if checkpoint.db is on the mounted volume.*

Human actor · User interface · Deterministic code / config · LLM-backed component · Trained ML model · Data store / external service · Gate / secrets · request / data flow · outbound call to an external service